

6SENG005W Formal Methods (2025/26) Coursework

Module Leader	Klaus Draeger (K.Draeger@westminster.ac.uk)
Unit	Coursework
Weighting:	50%
Qualifying mark	30%
Description	To develop a B specification of a version of the <i>Battleships</i> board game using the B tools Atelier B and ProB.
Learning Outcomes Covered in this Assignment:	The coursework assesses learning outcomes: LO1, LO2, LO3 & LO4.
Handed Out:	October 2025
Due Date	13:00, Tuesday, 6th January, 2026
Expected deliverables	Battleships board game B Specification: B machine(s) (.mch)
Method of Submission:	Online via Blackboard
Type of Feedback and Due Date:	Written feedback and marks available within 15 working days of submission. All marks will remain provisional until formally agreed by an Assessment Board.

Assessment regulations

Refer to section 4 of the “How you study” guide for undergraduate students for a clarification of how you are assessed, penalties and late submissions, what constitutes plagiarism etc.

Penalty for Late Submission

If you submit your coursework late but within 24 hours or one working day of the specified deadline, 10 marks will be deducted from the final mark, as a penalty for late submission, except for work which obtains a mark in the range 40 – 49%, in which case the mark will be capped at the pass mark (40%). If you submit your coursework more than 24 hours or more than one working day after the specified deadline you will be given a mark of zero for the work in question unless a claim of Mitigating Circumstances has been submitted and accepted as valid.

It is recognised that on occasion, illness or a personal crisis can mean that you fail to submit a piece of work on time. In such cases you must inform the Campus Office in writing on a mitigating circumstances form, giving the reason for your late or non-submission. You must provide relevant documentary evidence with the form. This information will be reported to the relevant Assessment Board that will decide whether the mark of zero shall stand. For more detailed information regarding University Assessment Regulations, please refer to the following website:

<http://www.westminster.ac.uk/study/current-students/resources/academic-regulations>

Coursework description

1. Introduction

This coursework requires you to develop a B specification of a version of the *Battleships* board game.

Battleships is a strategy type guessing game for two players. It is played using two 10 x 10 grids, one for each player. The game starts by each player placing the ships of their fleet on squares in their own grid. The locations of the fleets are concealed from the other player. Players take alternate turns "*shooting*" at the other player's ships in an attempt to sink them. The aim of the game is to win by destroying your opponent's fleet of ships before they destroy yours.

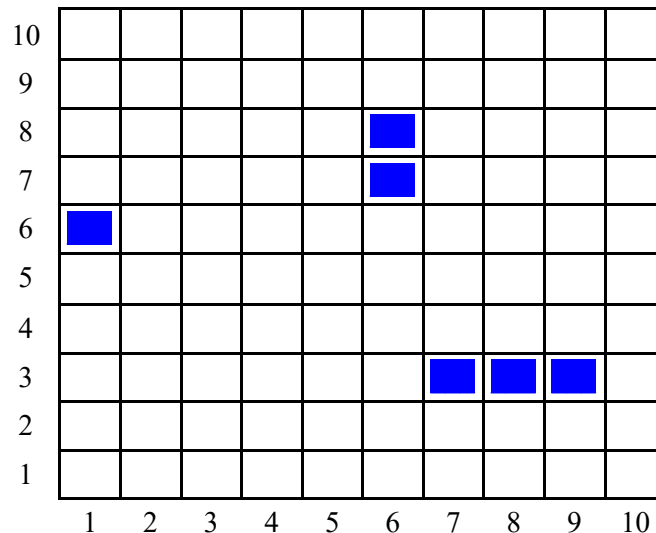


Figure 1. Example of a Player's Battleships Board with 3 ships

For example, if Player 1 has their fleet positioned as in Figure 1, then if Player 2 "*shoots*" at square (8,3), they score a "*hit*" and possibly sink the ship in that position (if they have previously hit (7,3) and (9,3)). If on Player 2's next turn, they "*shoot*" at square (4,4) then it is a "*miss*" and has no effect on Player 1's ships.

The *winner* of the game is the first player who sinks all their opponent's ships.

1.1 The Battleships Game Constraints and Requirements

For the purposes of this coursework, the Battleships game used is a simplified version of the normal game:

- A grid size of 10 x 10 squares, with the grid square (6,8) representing the square at column 6 and row 8.
- Each player has 3 ships in their fleet:
 - A **submarine**, occupying a single square
 - A **destroyer**, occupying 1*2 squares (horizontally or vertically)
 - A **cruiser**, occupying 1*3 squares (horizontally or vertically)
- A player's ships cannot have squares in common, i.e. at most 1 ship per square. It is however possible that the same square is used by a player 1 ship and a player 2 ship (since they use the grid to represent different areas of the sea).

1.2 Playing the Battleships Game

The Battleships game is played as follows:

1. The game starts by each player placing their 3 ships on the grid
 1. Each player places their submarine first, then their destroyer and finally their cruiser.
 2. There is no fixed order between the players. For example, Player 1 could place all their ships first followed by Player 2, or they could take turns, or anything in between.
2. Players take turns trying to sink each other's ships, starting with Player 1:
 1. Player 1 shoots at Player 2's ships, by selecting a target square on Player 2's grid. If there is a ship in the target square it is a hit and possibly sinks the ship, otherwise it is a miss.
 2. Player 2 shoots at Player 1's ships, by selecting a target square on Player 1's grid. Registering a hit or miss, as above.
3. The players continue to take turns until all of one player's ships have been sunk, at this point the player who has at least 1 ship left is the winner.

2. The Battleships Game B Specification

2.1 Specification Components

Your B specification should include the following elements.

(a) Sets and Constants

Any sets and constants that are required to define the data and state of the Battleships game system, and their properties. Think about how to represent the **players** and **their fleets** as well as the **possible game states**.

(b) System State

The state variables required to represent the data for the Battleships game system, including the state invariant and initialisation.

(c) Operations

The four Battleships game operations are:

```
report <-- DeploySubMarine( player, position )
```

The player tries to place their submarine at the given position. This is only possible if this **player's submarine has not been previously placed**.

If the position is **valid**, the operation **reports success**, otherwise **it reports an error message** indicating what the error was.

```
report <-- DeployDestroyer( player, position, orientation )
```

The player tries to place their destroyer at the given position, with the **given orientation** (**across** or **down**; the **position** gives the **top/left cell**). This is only possible if this player's **submarine has been previously placed** but the **destroyer hasn't**.

If the **position is valid**, and there is **no overlap**, then the operation **reports success**, otherwise it **reports an error message** indicating what the error was.

```
report <-- DeployCruiser( player, position, orientation )
```

The player tries to place their cruiser at the given position, with the **given orientation** (**across or down**; the **position** gives the **top/left** cell). This is only possible if this player's **destroyer has been previously placed** but the **cruiser hasn't**.

If the **position is valid**, and there is **no overlap**, then the operation **reports success**, otherwise it **reports an error message** indicating what the error was.

```
report <-- Shoot( player, target )
```

The player shoots at the target square on their opponent's grid. This is only possible if **setup has been completed** (i.e. both cruisers have been placed).

If there is a **ship in the target square**, then either the **ship is sunk** (if all its squares have **been hit**) and the operation **reports sunk**, or **it isn't**, and the operation **reports a hit**; if the **hit ship is the last one in the opponent's fleet** the **game is over** and the **game state updated accordingly**. If there is **no ship in the target square** the operation **reports a miss**. If the **target square is not in the grid**, it **reports an error message**.

Note that all operations must output the outcome of an operation, i.e. successful or if it fails, the reason why it failed with an appropriate error message.

(d) Enquiry Operations

You must also specify the following enquiry operations, that all output information about the state of the game:

```
shipSquares <-- LocationOfShips( player )
```

Outputs the locations of all the ships in the player's fleet.

```
shipsCount <-- ShipsLeft( player )
```

Outputs the number of ships left in the player's fleet.

```
shotCount <-- ShotsTaken( player )
```

Outputs the number of shots taken by the player.

```
report <-- GameState
```

Returns the game state, e.g. setting up, playing, Player 1 win ...

2.2 General Requirements

The B specification should use the appropriate features to define the data and operations in any machines that you define. The overall structure of the B specification can be either:

- A single B machine that contains all the state information and operations, etc.

- A collection of B machines that together contain all of the state information and operations, etc. The appropriate B structuring features must be used to combine the collection of B machines.

The specification must be syntactically and type correct, as checked by using the Atelier B tool. The specification must work when simulated in ProB. That is, it must initialise correctly and all operations must work as specified above to place ships and shoot at ships.

2.3 Blackboard Submission

Upload either a single mch file (if using one machine) or a zip file containing all mch files (if using multiple machines) directly on blackboard.

Marking Scheme Overview

The Coursework will be marked based on the following main component marking criteria, the full marking criteria details will be published on the module Blackboard site.

Criteria	Mark per component
Sets and constants	15
Variables, invariant and initialisation	15
DeploySubmarine operation	10
DeployDestroyer operation	10
DeployCruiser operation	10
Shoot operation	20
LocationOfShips operation	5
ShipsLeft operation	5
ShotsTaken operation	5
GameState operation	5
Total	100